

深圳大学实验报告

课程名称: 计算机系统 1

实验项目名称: 实验四

学院: 计算机与软件学院

专业: 计算机科学与技术

指导教师: 李志

报告人: 汉雨 学号: 2024040042 班级: 国际班

实验时间: 20260607

教务处制

一、实验目的

本实验要求在 **LC-3** 环境中实现一个 6×6 的简易四子棋。程序需要完成 **棋盘显示、输入校验、落子更新、胜负判断、平局判断** 与终局输出等功能。

两名选手轮流输入列号 1-6，棋子从该列最底部开始落下；若列已满或输入非法，程序给出提示并要求重新输入。任意一方在横向、纵向或两条对角线方向形成连续四子时获胜；若 36 个位置均被填满且无人获胜，则判定为平局。

规则要点

- 使用 '-' 表示空格，'O' 表示 Player 1，'X' 表示 Player 2。
- 玩家只能输入列号，不输入行号；程序自动寻找该列最低的空位。
- 非法字符、越界列号或满列输入都不会改变棋盘状态。
- 每次成功落子后立即重新显示棋盘，并检查胜负或平局。

输出样例：初始 6×6 棋盘

```
- - - - -  
- - - - -  
- - - - -  
- - - - -  
- - - - -  
- - - - -
```

二、实验内容与思路

核心设计

用 BOARD 的 36 个连续内存单元表示棋盘；用 0、1、-1 分别表示空格、Player 1 与 Player 2。这样在判断胜负时，只要统计连续 4 个位置的数值和，结果为 4 或 -4 就表示同一玩家四子连线。

BOARD	保存 6×6 棋盘，按行连续存放，共 36 个字。
TIMES	记录当前回合数；既用于判断玩家身份，也用于平局检测。
DISPLAY	遍历 BOARD，将 0/1/-1 转换为 -/O/X 并打印棋盘。
INPUT	读取键盘字符并判断是否为 1-6 的有效列号。
PLAY	从指定列底部向上寻找空位，完成落子并更新 TIMES。
JUDGE_W	从横、竖、两条斜线四个方向扫描是否存在连续四子。
ANSWER	根据游戏结束原因输出获胜玩家或 Tie Game。

三、子程序设计与关键代码

本节保留核心逻辑代码，并对重复结构作适当删减。每段代码上方先说明流程，代码表格中补充中文注释；其中黄色标注和红色文字用于强调容易出错或影响结果的关键语句。

1. 主程序 MAIN：控制整局游戏循环

主程序负责串联所有子程序：先打印棋盘，再提示玩家输入；输入通过后调用落子逻辑，成功落子后立即检查胜负或平局。若游戏未结束，则回到循环开头；若结束，则输出最终棋盘和结果。

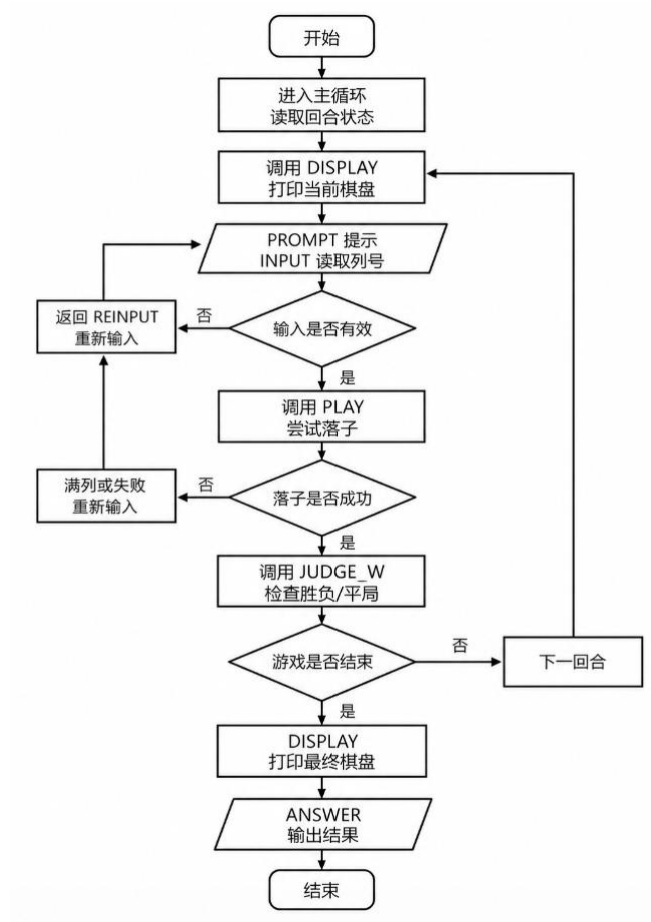


图 3-1 主程序 MAIN 流程图

代码片段：主程序循环

```

.ORG x3000
LOOP1  LD  R0, TIMES      ;读取回合计数，决定当前玩家
        JSR DISPLAY      ;打印当前棋盘
REINPUT JSR PROMPT        ;重要：先提示当前玩家输入列号
        JSR INPUT        ;读取并校验 1-6
        ADD R0, R0, #0
        BRz REINPUT      ;非法输入则重新输入
        JSR PLAY         ;落子并更新棋盘
        ADD R0, R0, #0
        BRz REINPUT      ;满列或无效落子时重新输入
        JSR JUDGE_W      ;重要：每次落子后立即判断胜负或平局
        ADD R0, R0, #0
        BRp OVER
        BRnzp LOOP1
OVER   JSR DISPLAY      ;输出最终棋盘
        JSR ANSWER       ;输出胜利或平局结果
        HALT
  
```

2. DISPLAY: 将棋盘数组转换为可视化棋盘

DISPLAY 从 BOARD 首地址开始逐格读取。每个单元根据数值转换为显示字符：0 输出 '-', 1 输出 'O', -1 输出 'X'。程序在列之间输出空格，每行结束后输出换行，从而形成整齐的 6×6 棋盘。

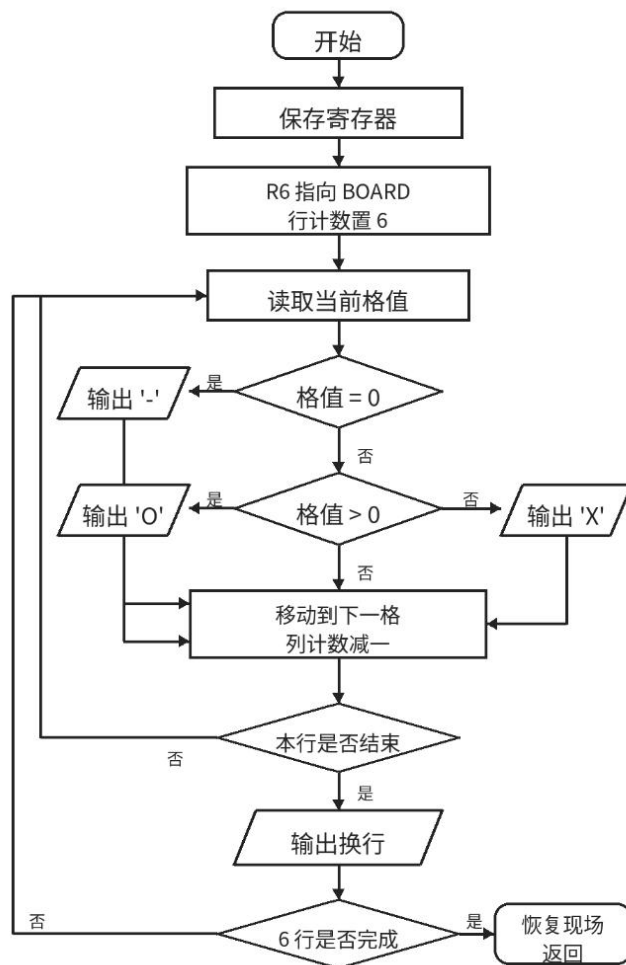


图 3-2 DISPLAY 子程序流程图

代码片段：DISPLAY 棋盘显示

```
DISPLAY ST R1, SAVE1      ;保护寄存器
        ST R2, SAVE2
        ST R3, SAVE3
        ST R6, SAVE6
        LEA R6, BOARD      ;R6 指向棋盘首地址
        LD R2, NUM_ROW     ;外层循环：6 行
COL_L   LD R1, NUM_ROW     ;内层循环：6 列
ROW_L   LDR R3, R6, #0     ;读取当前格子的状态
        BRnp TYPE_L1
        LD R0, CHESS_N     ;0 显示为 '-'
        BRnzp PRINT_CH
TYPE_L1 BRp TYPE_L2
        LD R0, CHESS_X     ;-1 显示为 X
        BRnzp PRINT_CH
TYPE_L2 LD R0, CHESS_O     ;1 显示为 O
PRINT_CH TRAP x21          ;逐字符输出棋盘
        ADD R6, R6, #1     ;移动到下一格
        ...               ;空格与换行逻辑相同，省略重复输出
        RET
```

3. ANSWER：输出终局结果

ANSWER 先判断棋盘是否已满，若已满则输出平局；否则根据 TIMES 的奇偶性判断最后一次成功落子来自哪位玩家，并输出对应的胜利信息。

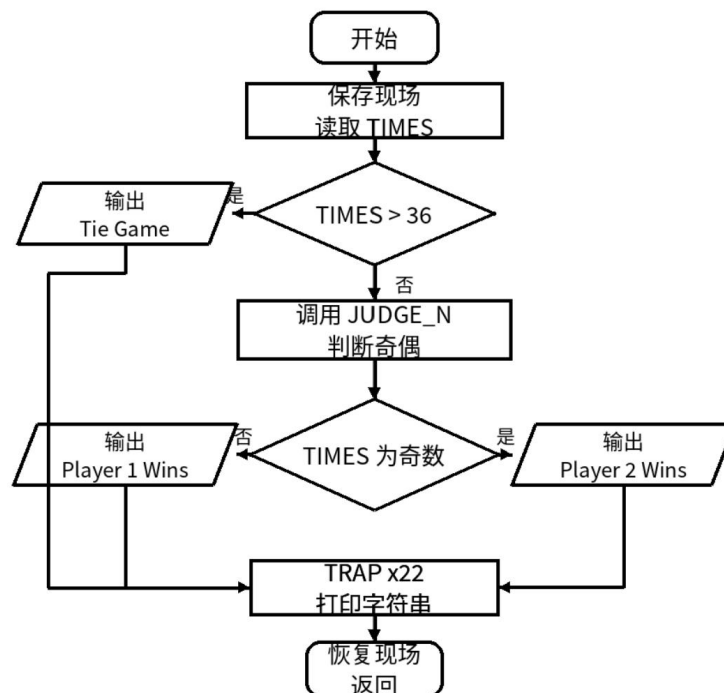


图 3-3 ANSWER 子程序流程图

代码片段：ANSWER 终局输出

```
ANSWER  ST R1, SAVE1      ;保护现场
        ST R2, SAVE2
        LD R1, TIMES
        LD R2, NUM_END    ;R2=-36, 用于判断棋盘是否已满
        ADD R2, R1, R2
        BRnz ANS_L1
        LEA R0, TIE        ;平局: 输出 Tie Game.
        BRnzp ANS_LD
ANS_L1  LD R0, TIMES
        JSR JUDGE_N        ;根据奇偶判断上一手属于哪位玩家
        BRp ANS_L2
        LEA R0, WIN1       ;输出 Player 1 Wins.
        BRnzp ANS_LD
ANS_L2  LEA R0, WIN2       ;输出 Player 2 Wins.
ANS_LD  TRAP x22
        RET
```

4. PROMPT：提示输入并处理错误提示

PROMPT 的职责是输出交互提示语。当上一轮输入无效时，先输出 Invalid move，再根据当前回合数决定提示 Player 1 还是 Player 2 输入列号。

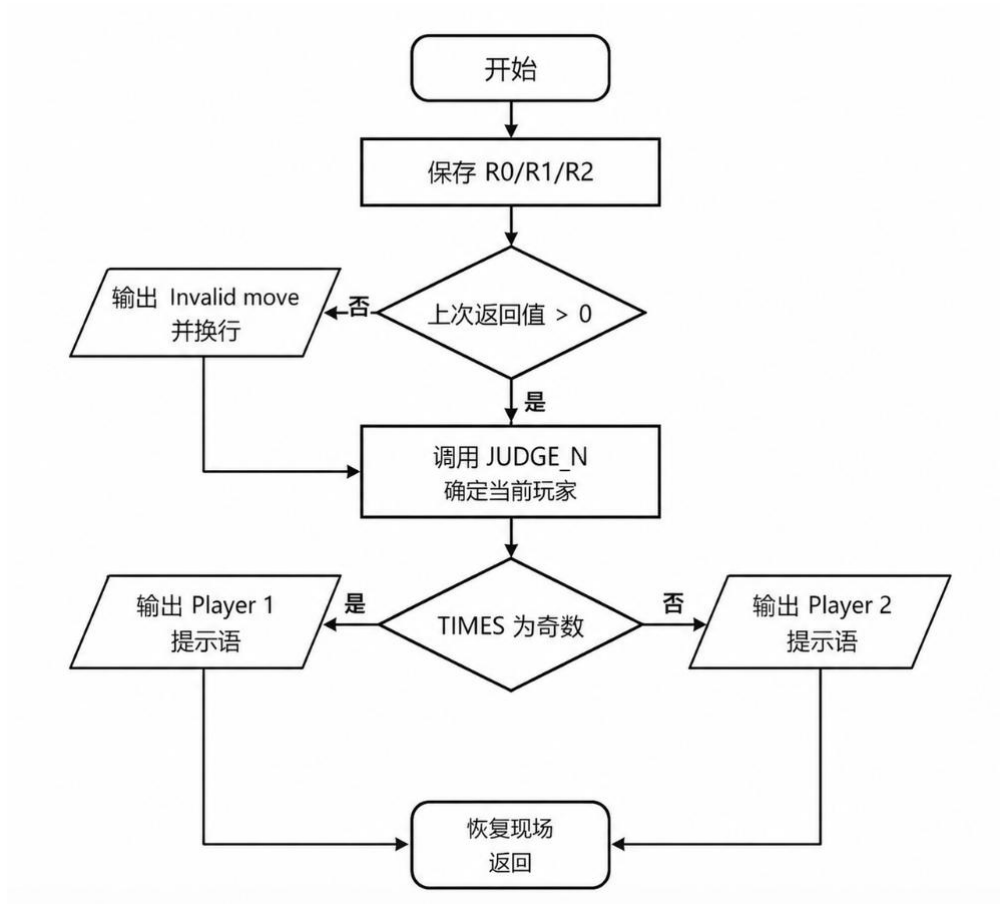


图 3-4 PROMPT 子程序流程图

代码片段：PROMPT 输入提示

```

PROMPT  ST R1, SAVE1      ;保护寄存器
        ST R2, SAVE2
        ST R0, SAVE0
        ADD R0, R0, #0
        BRp PRO_L3
        LEA R0, INVALID   ;非法输入时先输出错误提示
        TRAP x22
PRO_L3  LD R0, TIMES
        JSR JUDGE_N        ;判断当前应提示 Player 1 还是 Player 2
        BRz PRO_L1
        LEA R0, PROMPT1
        BRnzp PRO_L2
PRO_L1  LEA R0, PROMPT2
PRO_L2  TRAP x22          ;输出交互提示语
        RET
    
```

5. 数据区 LABEL：字符串、棋子与状态常量

数据区集中保存提示字符串、棋子字符、棋盘数组和常量。把这些数据统一放置，能让各子程序只通过标签读取，减少硬编码并提升可维护性。



图 3-5 LABEL 数据区组织流程图

代码片段：LABEL 与全局数据

```
TIE      .STRINGZ "Tie Game."
INVALID  .STRINGZ "Invalid move. Try again. "
PROMPT1  .STRINGZ "Player 1, choose a column: "
PROMPT2  .STRINGZ "Player 2, choose a column: "
WIN1     .STRINGZ "Player 1 Wins."
WIN2     .STRINGZ "Player 2 Wins."

CHESS_O  .FILL x004F      ;O 棋子的 ASCII 码
CHESS_X  .FILL x0058      ;X 棋子的 ASCII 码
CHESS_N  .FILL x002D      ;空格显示为 '-'
TIMES    .FILL #1         ;重要：回合计数，同时用于判断平局
BOARD    .BLKW 36 #0      ;核心：36 个连续单元模拟 6x6 棋盘
NUM_ROW  .FILL #6
NUM_END  .FILL #-36
NUM_CH   .FILL #-48      ;将 ASCII 数字转为整数列号
```

6. INPUT：读取并校验列号

INPUT 读取一个键盘字符后，先通过减去 ASCII '0' 的偏移量将其转换为数字；随后检查该数字是否位于 1-6。若不合法，R0 被清零，主程序据此重新提示输入。

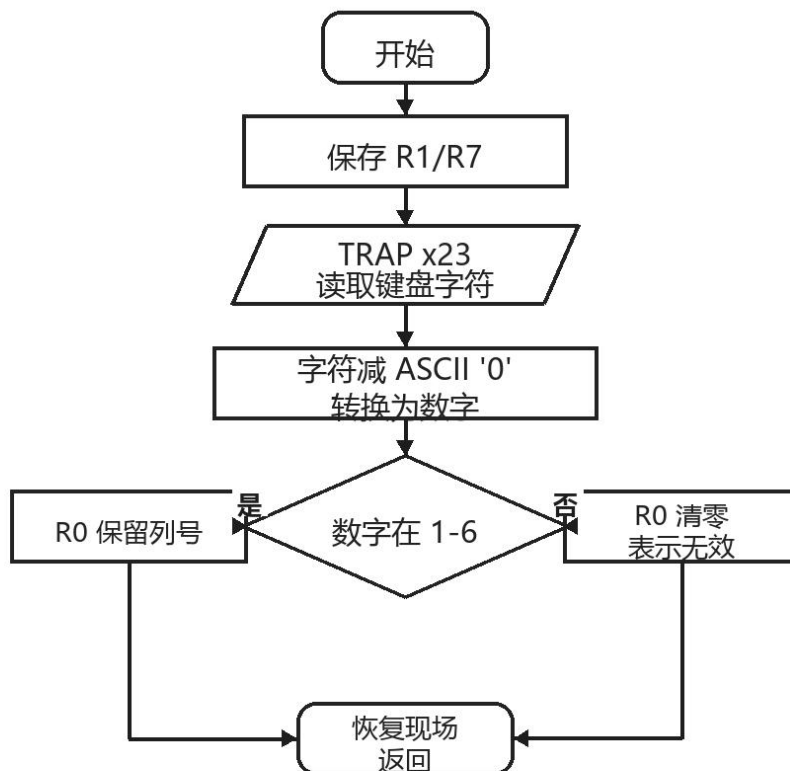


图 3-6 INPUT 子程序流程图

代码片段：INPUT 输入校验

```

INPUT  ST R1, SAVE1      ;保护 R1
        ST R7, SAVE7
        TRAP x23          ;从键盘读取一个字符
        LD R1, NUM_CH     ;R1=-48, 将 ASCII 转为数字
        ADD R0, R0, R1
        BRnz INP_L1       ;非法: 小于 1
        AND R1, R1, #0
        ADD R1, R1, #-6
        ADD R1, R0, R1
        BRp INP_L1        ;非法: 大于 6
        BRnzp INP_L2
INP_L1  AND R0, R0, #0     ;非法输入统一返回 0
INP_L2  LD R1, SAVE1
        RET
  
```

7. PLAY: 完成落子与棋盘更新

PLAY 根据输入列号定位到该列最底部，然后自下而上寻找第一个空位。找到空位后，根据当前玩家写入 1 或 -1，并将 TIMES 加一；若整列已满，则返回 0，主程序会要求玩家重新输入。

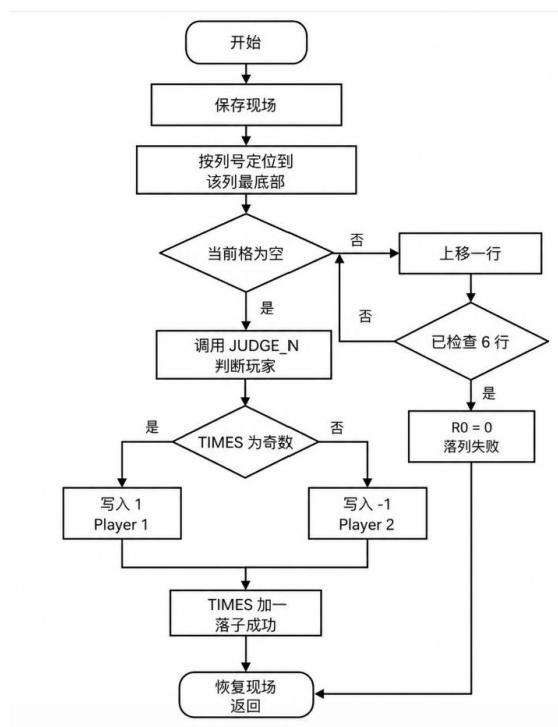


图 3-7 PLAY 子程序流程图

代码片段：PLAY 落子逻辑

```
PLAY      ST R1, SAVE1      ;保护现场
          ST R2, SAVE2
          ST R3, SAVE3
          AND R3, R3, #0
          ADD R3, R3, #6     ;从最底行开始查找可落子位置
          LEA R1, BOARD
          ADD R1, R1, #29
          ADD R1, R1, R0     ;定位到指定列的底部
PLA_L4    LDR R2, R1, #0
          BRz PLA_L3        ;找到空位即可落子
          ADD R3, R3, #-1
          BRnz PLA_LD       ;满列：返回 0 让用户重输
          ADD R1, R1, #-6   ;向上一行继续搜索
          BRnzp PLA_L4
PLA_L3    JSR JUDGE_N       ;根据回合奇偶确定写入 1 或 -1
          BRz PLA_L2
          ADD R2, R2, #1    ;Player 1 写入 1
          BRnzp PLA_L1
PLA_L2    ADD R2, R2, #-1   ;Player 2 写入 -1
PLA_L1    STR R2, R1, #0   ;重要：真正更新棋盘数组
          ...              ;TIMES 加一并恢复寄存器
          RET
```

8. JUDGE_N 与 SUM_N：辅助判断奇偶与连续四格

JUDGE_N 用 TIMES 的最低位判断当前回合奇偶；SUM_N 则按照给定步长连续读取 4 个位置并求和。由于棋盘状态采用 1 与 -1 表示玩家，连续四个同色棋子的和一定为 4 或 -4。

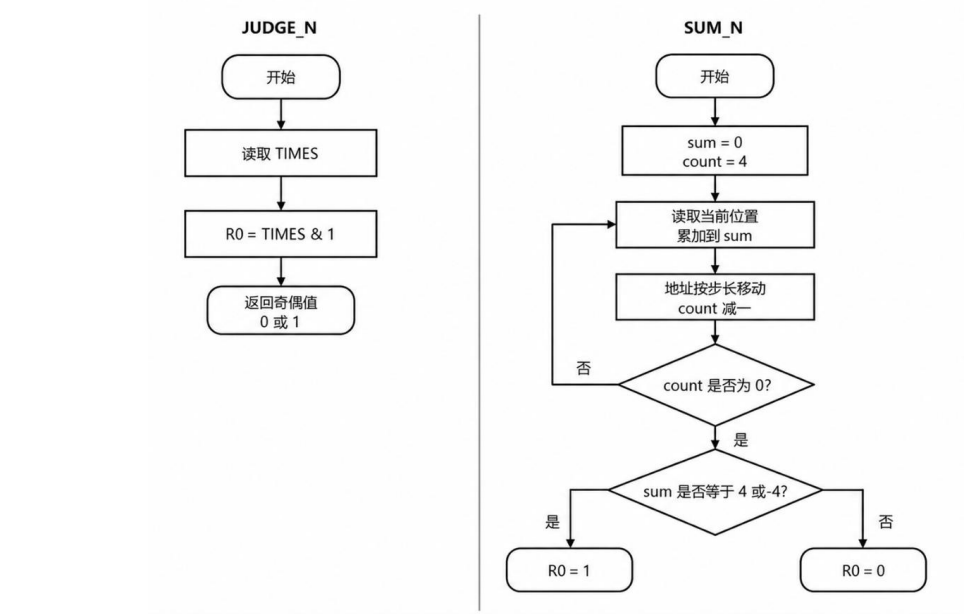


图 3-8 JUDGE_N 与 SUM_N 辅助子程序流程图

代码片段：JUDGE_N / SUM_N 辅助判断

JUDGE_N LD R1, TIMES;读取回合计数

AND R0, R0, #0

ADD R0, R0, #1

AND R0, R0, R1;R0=1 表示奇数回合, R0=0 表示偶数回合

RET

SUM_N ST R1, SAVE1;保护寄存器

ST R2, SAVE2

ST R3, SAVE3

AND R3, R3, #0;R3 保存连续 4 格的和

ADD R2, R2, #4

SUM_L1 LDR R1, R6, #0

ADD R3, R1, R3

ADD R6, R6, R0;R0 是扫描步长: 1/5/6/7

ADD R2, R2, #-1

BRp SUM_L1

...;若和为 4 或 -4, 则说明同一玩家四子连线

RET

9. JUDGE_W: 四方向胜负与平局判断

JUDGE_W 统一调用 SUM_N 完成扫描。横向步长为 1，纵向步长为 6，左下斜线步长为 7，右上斜线步长为 5。任一方向发现连续四子即返回正数；若没有获胜但回合数超过棋盘容量，则返回平局结束信号。

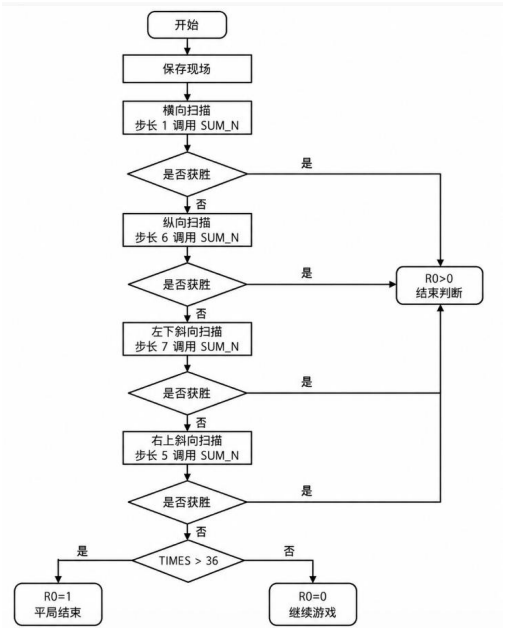


图 3-9 JUDGE_W 子程序流程图

代码片段：JUDGE_W 胜负判断

```
JUDGE_W ST R1, SAVE1      ;保护现场
        ST R2, SAVE2
        ST R6, SAVE6
        ; 横向检测：起点逐格右移，步长为 1
        LD R1, NUM_ROW
        LEA R6, BOARD
WIN_L1  AND R0, R0, #0
        ADD R0, R0, #1      ;步长 1：横向连续 4 格
        JSR SUM_N
        BRp WIN_LW          ;重要：发现四子连线立即结束
        ...                 ;纵向改用步长 6，两条斜线改用步长 7 和 5
        ; 四个方向均未获胜后，再检查是否达到 36 手
        LD R1, TIMES
        LD R2, NUM_END
        ADD R2, R1, R2
        BRnz WIN_LD
        ADD R0, R0, #1      ;平局也作为游戏结束返回正数
WIN_LD  LD R1, SAVE1
        LD R2, SAVE2
        LD R6, SAVE6
        RET
```

四、实验步骤与结果

1. 初始界面与等待输入

程序启动后首先打印空棋盘，并提示 Player 1 输入列号。此时棋盘 36 个位置均为空，显示为 '-', 交互流程符合实验要求。

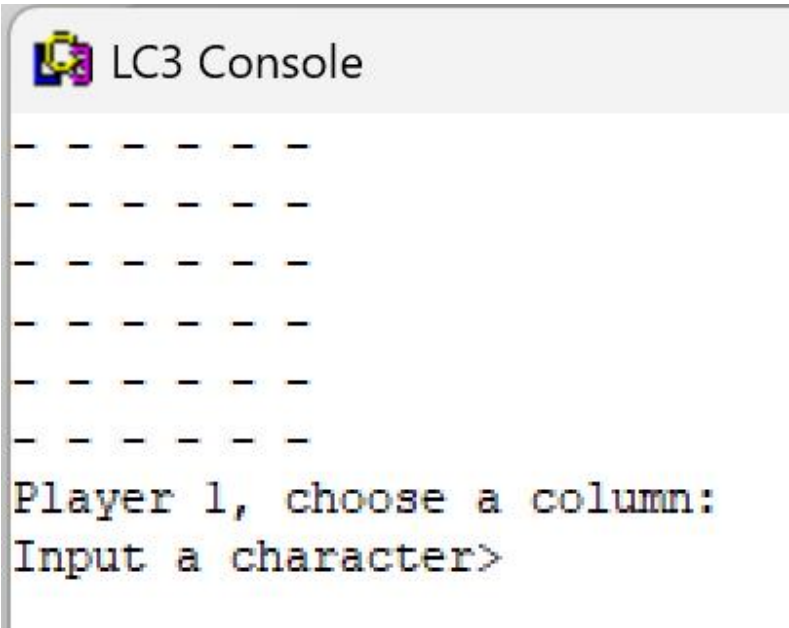


图 1 初始棋盘与等待用户输入界面

2. 非法输入与满列处理

当输入字符不是 1-6 的有效列号，或对应列已经没有可落子空间时，程序输出 Invalid move. Try again., 并保持棋盘状态不变。这说明 INPUT 与 PLAY 的错误返回机制能够被主循环正确捕获。

```
LC3 Console
- - - - -
- - - - -
- - - - -
- - - - -
- - - - -
- - - - -
- - - - -
- - - - -
Player 1, choose a column:
Input a character>s
Invalid move. Try again.
Player 1, choose a column:
Input a character>
```

图 2 非法字符输入后的提示

```
LC3 Console
- - - - -
- - - - -
- - - - -
- - - - -
- - - - -
- - - - -
- - - - -
- - - - -
Player 1, choose a column:
Input a character>0
Invalid move. Try again.
Player 1, choose a column:
Input a character>
```

图 3 非法列号输入后的重新提示

```
Input a character>7
Invalid move. Try again.
Player 1, choose a column:
Input a character>
```

图 4 非法列号输入后的重新提示

```
-
Player 2, choose a column:
Input a character>1
X - - - -
O - - - -
X - - - -
O - - - -
X - - - -
O - - - -
Player 1, choose a column:
Input a character>1
Invalid move. Try again.
```

图 5 满列后拒绝继续落子

3. 平局情况

当棋盘被填满且没有任何方向形成连续四子时，程序输出 Tie Game. 并终止。该测试覆盖了 TIMES 与 NUM_END 配合判断棋盘容量的逻辑。

```
LC3 Console
X O X O X O
O X O X O X
O X O X O X
O X O X O X
Player 1, choose a column:
Input a character>6
X O X O - O
X O X O X O
X O X O X O
O X O X O X
O X O X O X
O X O X O X
Player 2, choose a column:
Input a character>5
X O X O X O
X O X O X O
X O X O X O
O X O X O X
O X O X O X
O X O X O X
Tie Game.
A trap was executed with an illegal vector number.
----- Halting the processor -----
```

图 6 棋盘填满后的平局输出

4. 横向连线获胜

横向测试中，同一玩家在同一行形成连续四个棋子，JUDGE_W 通过步长为 1 的扫描调用 SUM_N，成功识别胜利并输出对应玩家获胜。

```
LC3 Console
-----
X X - - -
O O O - - -
Player 2, choose a column:
Input a character>3
-----
X X X - - -
O O O - - -
Player 1, choose a column:
Input a character>4
-----
X X X - - -
O O O O - -
Player 1 Wins.
----- Halting the processor -----
```

图 7 横向四子连线获胜

5. 纵向连线获胜

纵向测试中，同一列中形成连续四子。程序通过步长为 6 的扫描跨行读取棋盘数组，验证了按列判断的地址跳转逻辑。

```
LC3 Console
- - - - -
- - - - -
O - - - -
O X - - -
O X - - -
Player 2, choose a column:
Input a character>2
- - - - -
- - - - -
- - - - -
O X - - -
O X - - -
O X - - -
Player 1, choose a column:
Input a character>1
- - - - -
- - - - -
O - - - -
O X - - -
O X - - -
O X - - -
Player 1 Wins.
----- Halting the processor -----
```

图 8 纵向四子连线获胜

6. 两种斜向连线获胜

斜向测试覆盖了左下与右上两个方向。两类斜线只是在起点与步长上不同，整体仍复用

SUM_N 的连续四格求和逻辑，说明胜负判断具有较好的复用性。

```
LC3 Console
- - - - -
- - - - -
- - - X - -
- - X X O -
O X O X O O
Player 1, choose a column:
Input a character>5
- - - - -
- - - - -
- - - X O -
- - X X O -
O X O X O O
Player 2, choose a column:
Input a character>4
- - - - -
- - - - -
- - - X - -
- - - X O -
- - X X O -
O X O X O O
Player 2 Wins.
----- Halting the processor -----
```

图 9 斜向左下四子连线获胜

```
LC3 Console
- - X - - -
- O X - - -
- O O X - -
- O X O X -
O X O X O X
Player 1, choose a column:
Input a character>6
- - - - -
- - X - - -
- O X - - -
- O O X - -
- O X O X O
O X O X O X
Player 2, choose a column:
Input a character>2
- - - - -
- X X - - -
- O X - - -
- O O X - -
- O X O X O
O X O X O X
Player 2 Wins.
----- Halting the processor -----
```

图 10 斜向右上四子连线获胜

五、实验结论

本实验完成了基于 **LC-3 汇编** 的简易四子棋程序设计。程序能够正确实现棋盘初始化、玩家轮流输入、非法输入拦截、满列检测、落子更新、四方向胜负判断以及平局输出。

通过测试可以看到，程序在等待输入、非法输入、平局、横向获胜、纵向获胜以及两种斜向获胜等典型场景下均能得到正确结果。实现过程中，BOARD 数组把二维棋盘映射到连续内存，TIMES 同时承担回合控制与终局判断，SUM_N 则把四个方向的胜负检测抽象为“起点 + 步长 + 连续四格求和”的统一模型。

收获与改进方向

本实验加深了对 LC-3 子程序调用、寄存器保护、内存寻址、条件分支和 TRAP 输入输出的理解。后续

若继续优化，可以增加列号显示、玩家落子历史提示，或进一步压缩 JUDGE_W 中不同方向扫描的重复代码，使程序结构更加简洁。

指导教师批阅意见：

成绩评定：

指导教师签字:

年 月 日

备注:

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。